

Graphs: Representation and the Two Standard Encodings

Programming and Data Structures — Week 11, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to define a graph precisely and place both trees and linked lists as special cases within this more general structure; implement both an adjacency list and an adjacency matrix from scratch; state the time and space cost of each representation for every core graph operation; and choose the correct representation for a concretely described graph, using edge density as the deciding factor.

Outline

Today covers a social-network analogy motivating graph structure; graphs defined precisely, with trees and linked lists placed as special cases within this more general structure; the directed-versus-undirected and weighted-versus-unweighted distinctions; the adjacency matrix, built from scratch; the adjacency list, built from scratch; a direct cost comparison determining when each representation wins; the MTech-track material on sparse versus dense graphs and realistic graph sizes; an in-class quiz; and a summary.

The social-network analogy

Picture people as nodes and friendships as edges connecting pairs of them. Unlike Week 7's binary trees, there is no single designated root, and a node can connect to any number of other nodes in any pattern whatsoever, including cycles — a group of mutual friends forms exactly such a cycle. This is the final generalization of structure covered in this course: Week 7's trees required a strict hierarchical shape, no node having more than one parent; a graph requires nothing more than a set of nodes and a set of connections between them, with no shape constraint imposed at all.

Graphs, defined precisely

A graph $G = (V, E)$ consists of a set of vertices V , the nodes, and a set of edges E , connections between pairs of vertices. This definition is deliberately general enough to subsume structures already covered: Week 7's binary trees are graphs specifically constrained to be connected and acyclic, with every node having at most one parent. Week 4's linked lists are graphs too, constrained even further, to at most one neighbor in each direction per node. Recognizing this is not merely a theoretical curiosity — every traversal algorithm this lecture and the next build directly generalizes traversal techniques already covered for trees.

Directed vs. undirected, weighted vs. unweighted

Graphs come in two independent binary distinctions. Undirected graphs treat an edge between u and v as implying a connection in both directions, like a mutual friendship. Directed graphs distinguish an edge from u to v from one in the reverse direction, like a one-way social media follow. Weighted graphs attach a cost or distance to each edge, such as a road's physical length; unweighted graphs treat every edge as equally significant. Week 10 Lecture 3's minimum spanning tree problem specifically used undirected, weighted graphs — the representations built in this lecture must support all four possible combinations of these two distinctions.

A concrete example graph, used throughout this lecture

Nodes: 0,1,2,3,4

Edges (undirected, unweighted): (0,1), (0,2), (1,2), (1,3), (3,4)

This small 5-node, 5-edge graph is used throughout the remainder of this lecture, deliberately kept small and sparse so both representations can be traced completely by hand rather than only described abstractly. Node 1 has the highest degree, with 3 neighbors (0, 2, and 3) — worth tracking specifically as both representations are constructed on the following slides, since it is the node where the two representations' trade-offs become most visible.

The adjacency matrix, built from scratch

```
class GraphMatrix:
    def __init__(self, n):
        self.n = n
        self.matrix = [[0] * n for _ in range(n)] # Week 1's 2D array, directly reused

    def add_edge(self, u, v, weight=1, directed=False):
```

```

        self.matrix[u][v] = weight
    if not directed:
        self.matrix[v][u] = weight

def has_edge(self, u, v):
    return self.matrix[u][v] != 0

```

The adjacency matrix is a plain $n \times n$ two-dimensional array, directly reusing Week 1 Lecture 1's row-major structure, where `matrix[u][v]` holds the weight of the edge between u and v , or a sentinel value like 0 if no edge exists. Checking whether an edge exists, `has_edge(u, v)`, costs $O(1)$, exactly Week 1's address-formula argument applied to a 2D array. The critical cost, however, is space: this representation always uses $O(n^2)$ memory, regardless of how many edges actually exist in the graph — even a graph with very few edges still pays for the full $n \times n$ grid.

The adjacency list, built from scratch

```

class GraphList:
    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]    # Week 4's list-of-lists idea

    def add_edge(self, u, v, weight=1, directed=False):
        self.adj[u].append((v, weight))
        if not directed:
            self.adj[v].append((u, weight))

    def has_edge(self, u, v):
        return any(neighbor == v for neighbor, weight in self.adj[u])

```

The adjacency list keeps, for each node, a list containing only its actual neighbors — structurally analogous to Week 8's hash table chaining, where each bucket holds only the entries that actually collided there, rather than reserving space for every possible key. Checking `has_edge(u, v)` now costs $O(\text{degree of } u)$, the number of neighbors u actually has, since the list must be scanned — genuinely worse than the matrix's $O(1)$ for this specific operation. The space cost, however, is $O(V + E)$, proportional to the actual number of vertices and edges present, not the matrix's fixed $O(n^2)$ regardless of edge count.

Building the matrix for this exact graph

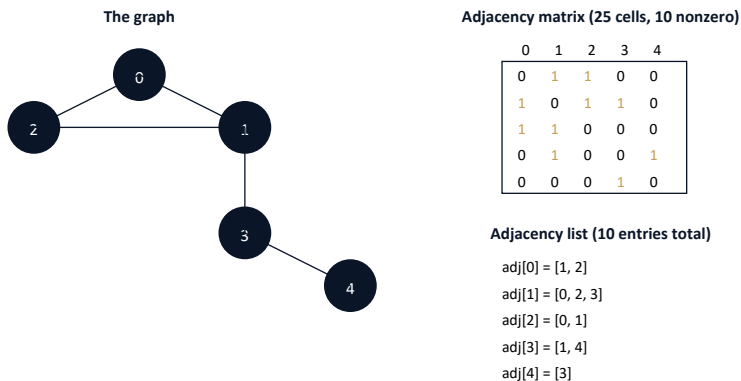
```
matrix[0][1]=1, matrix[1][0]=1 (edge 0-1)
matrix[0][2]=1, matrix[2][0]=1 (edge 0-2)
matrix[1][2]=1, matrix[2][1]=1 (edge 1-2)
matrix[1][3]=1, matrix[3][1]=1 (edge 1-3)
matrix[3][4]=1, matrix[4][3]=1 (edge 3-4)
```

Resulting 5x5 matrix:

```
    0  1  2  3  4
0 [ 0  1  1  0  0 ]
1 [ 1  0  1  1  0 ]
2 [ 1  1  0  0  0 ]
3 [ 0  1  0  0  1 ]
4 [ 0  0  0  1  0 ]
```

Building the matrix for this lecture's example graph completely: each undirected edge fills two symmetric cells, since `add_edge` writes both `matrix[u][v]` and `matrix[v][u]` when `directed=False`. The resulting matrix is symmetric across its diagonal, a general property of every undirected graph's adjacency matrix, and the diagonal itself is all zeros, since no self-loops exist in this example.

The picture: the same graph, two representations



The matrix reserves all 25 cells; the list stores only the 10 edge-entries that actually exist.

The matrix reserves a cell for every possible pair of nodes, most of them zero for a sparse graph like this one; the list stores only the edges that genuinely exist, making the wasted space in the matrix version directly visible by comparison.

Building the list for the same exact graph

```
add_edge(0,1): adj[0]=[(1,1)], adj[1]=[(0,1)]
add_edge(0,2): adj[0]=[(1,1),(2,1)], adj[2]=[(0,1)]
add_edge(1,2): adj[1]=[(0,1),(2,1)], adj[2]=[(0,1),(1,1)]
add_edge(1,3): adj[1]=[(0,1),(2,1),(3,1)], adj[3]=[(1,1)]
add_edge(3,4): adj[3]=[(1,1),(4,1)], adj[4]=[(3,1)]
```

Final adjacency lists:

```
adj[0] = [(1,1),(2,1)]
adj[1] = [(0,1),(2,1),(3,1)]
adj[2] = [(0,1),(1,1)]
adj[3] = [(1,1),(4,1)]
adj[4] = [(3,1)]
```

Building the adjacency list for the identical graph: each of the 5 edges appends exactly two entries total (one per direction, since the graph is undirected), giving 10 total list entries across all 5 nodes — directly comparable against the matrix's 25 cells (5×5), of which only 10 were non-zero. This concrete count — 10 meaningful entries versus 25 total cells, 15 of them wasted zeros — is exactly the wasted-space argument the matrix-versus-list comparison makes abstractly, now shown on a specific, traceable example.

Cost comparison: when each representation wins

This table consolidates the core trade-off directly. The matrix wins decisively on `has_edge` checks, $O(1)$ regardless of graph size, but loses badly on listing a node's neighbors, since it must scan an entire row of size n even if the node has very few actual neighbors. The list wins on both space and neighbor-listing when the graph has relatively few edges, but loses on `has_edge` checks, which must scan a variable-length neighbor list rather than perform a single lookup.

MTech addition — sparse vs. dense graphs, and real-world sizes

A graph is called sparse if its edge count E grows only linearly or near-linearly with the number of vertices V — most pairs of nodes are simply not connected, which describes nearly every real-world graph: social networks, road networks, and the hyperlink structure of the web are all genuinely sparse. A graph is called dense if E approaches V^2 , meaning most pairs of nodes are connected — this is rare in practice, though common in small synthetic textbook examples like this lecture's traced graphs. For a sparse graph with one million vertices, an adjacency matrix would require 10^{12} cells, computationally absurd regardless of available memory; an adjacency list requires space proportional only to the actual, far smaller edge count — this is why real-world graph libraries default to adjacency lists almost universally.

Degree, and its role in cost analysis

The degree of a node is the number of edges connected to it — node 1 in this lecture’s running example has degree 3, matching its three adjacency-list entries. A fundamental identity holds for any undirected graph: the sum of all nodes’ degrees always equals exactly $2|E|$, since every edge is counted once from each of its two endpoints. This identity is precisely why the adjacency list’s total space is $O(V + E)$ rather than something larger: the combined length of every node’s neighbor list sums to exactly $2|E|$, not $|V| \times |E|$ — confirmed directly on this lecture’s example, where degrees $2 + 3 + 2 + 2 + 1 = 10 = 2 \times 5$ matches the 10 total list entries counted on the previous slide.

Exercise

As an exercise, implement both `GraphMatrix` and `GraphList` for a 6-node graph of your own construction, and confirm that `has_edge` agrees between the two representations for every possible pair of nodes, checked exhaustively. Separately, for your constructed graph, compute the actual memory-proportional cost of each representation — counting matrix cells against total adjacency-list entries — and compare the two directly, connecting the numeric result back to this lecture’s sparse-versus-dense discussion.

Directed graphs: one asymmetric change

```
g_directed = GraphList(5)
g_directed.add_edge(0, 1, directed=True)    # only adj[0] gets an entry - NOT adj[1]
```

For a directed graph, `add_edge` deliberately skips the reverse insertion — an edge from 0 to 1 does not imply the reverse relationship exists. The adjacency matrix for a directed graph is, in general, no longer symmetric across its diagonal: `matrix[0][1]` can be 1 while `matrix[1][0]` remains 0, directly encoding the one-way relationship. Every other aspect of both representations — cost analysis, the traversal algorithms covered next lecture — generalizes identically to directed graphs; the only structural change is this one asymmetric insertion rule.

Quiz — choose the representation

Given a graph with 100,000 nodes and roughly 300,000 edges, where each node connects to only a handful of others, determine before checking the answer whether an adjacency matrix or an adjacency list is the appropriate representation, and why.

Quiz — answer

The adjacency list is the correct choice. This graph is clearly sparse: $E \approx 300,000$ is vastly smaller than $V^2 = 10,000,000,000$ for $V = 100,000$. An adjacency matrix would require 10 billion cells, computationally absurd; an adjacency list requires space proportional only to the roughly 300,000 actual edges, many orders of magnitude smaller. The detail that each node connects to only a handful of others is exactly the sparse-graph signature identified in this lecture's MTech addition, and is the concrete tell that should trigger choosing a list over a matrix.

A closing connection: adjacency list as “array of stacks/queues”

It is worth noticing that `self.adj = [[] for _ in range(n)]` is structurally an array, directly reusing Week 2, whose elements are themselves plain Python lists — one per node. Each inner list could equally be replaced with Week 3's Stack or Queue if a specific application needed neighbor order to matter, though a plain list suffices for most graph algorithms, since neighbor order is usually irrelevant to correctness. This is a quiet instance of Week 8 Lecture 4's “combine two structures” idea already present in this lecture's adjacency list: an outer array indexed by node, holding a per-node collection — the same compositional pattern, recurring again.

Summary

A graph generalizes every structure covered earlier in this course — both Week 7's binary trees and Week 4's linked lists are graphs with additional structural constraints imposed. The adjacency matrix offers $O(1)$ edge existence checks at the cost of $O(n^2)$ space regardless of how many edges actually exist. The adjacency list offers $O(\text{degree})$ edge checks but only $O(V + E)$ space, winning decisively for the sparse graphs that most real-world data actually produces. Choosing correctly between them depends on a graph's density, exactly the pattern-recognition discipline built since Week 8 Lecture 4's structure-choice framework. Next lecture introduces graph traversal — breadth-first and depth-first search — generalizing Week 7's tree traversal orders to structures that, unlike trees, may contain cycles.